

Liberta: A Hybrid Transformer-Linguistic Model for Machine-generated Text Detection

Fabian Kobylanskyj, Barış Şimşek, Jian Sun and Jiwoo Lee
Technical University of Munich, Germany

Abstract

With the rapid advancement of Large Language Models (LLMs) capable of generating text highly resembling human writing, a critical task is to distinguish between human written and machine-generated text. This paper describes our work on the SemEval 2024 Task 8: *Multidomain, Multimodel and Multilingual Machine-Generated Text Detection*. Our method focused on combining RoBERTa embeddings with readability and token-probabilistic features on tackling both the mono- and multilingual track of Subtask A: Human vs Machine Text Classification. On the official test dataset, our system achieved an accuracy of 88.64% in the monolingual and 79.87% in the multilingual track. Our results demonstrate that a hybrid approach can significantly enhance the performance in machine-generated text detection compared to a system that solely relies on a transformer-encoder.

1 Introduction

LLMs have a significant influence on our daily tasks across a wide range of domains, from education and research to content creation and communication. Since their impact on society is significantly shaping how we live and work, the potential risks associated with them are equally substantial. From spreading misinformation (Kreps et al., 2022), gaining unfair advantages in exams to plagiarism (Wahle et al., 2022), and a societal risk when it becomes unclear if communication or information comes from a human or machine, there are many dangers and potentially malicious use cases that make it crucial to develop automated means for detecting machine-generated text. In response, different detection tools have been developed. However, since a great deal of effort is put into developing and improving LLMs, the detection of machine-generated text turned out to be an ongoing arms race where the rapid improvement

of LLMs challenges detectors to deliver accurate results. A striking example for this is the OpenAI classifier, which was released in 2023 and discontinued shortly after due to a low rate of accuracy (Jan Hendrik Kirchner, 2023).

The SemEval 2024 Task 8: *Multidomain, Multimodel and Multilingual Machine-Generated Text Detection* addresses this challenge and provides an incentive to find solutions to the detection of machine-generated text across multiple generators (Wang et al., 2024). In the following we describe our solution to the monolingual track of Subtask A: Human vs Machine Text Classification, which is a hybrid model that combines RoBERTa embeddings, readability features and token-level probabilistic features. We also present and discuss the adaption of this hybrid model to the multilingual track of Subtask A, which additionally to English includes Arabic, German and Italian as surprising languages. During the development of our models we adhered to the same conditions as participating teams in the shared task in order to ensure a fair comparison. Our results suggest that a hybrid model that combines those three feature sets can significantly improve classification performance and yield a more robust approach than relying on a PLM alone. While RoBERTa successfully captures semantic nuances of the input text, the engineered features provide complementary information that RoBERTa alone misses.

2 Related Work

For the binary classification task of machine-generated text detection as in Subtask A, there are two main approaches. Zero-shot methods and supervised methods (Wang et al., 2024). Zero-shot methods have the advantage of not relying on curated datasets of human written and machine-generated text. This can be particularly beneficial in the context of the previously mentioned "arms

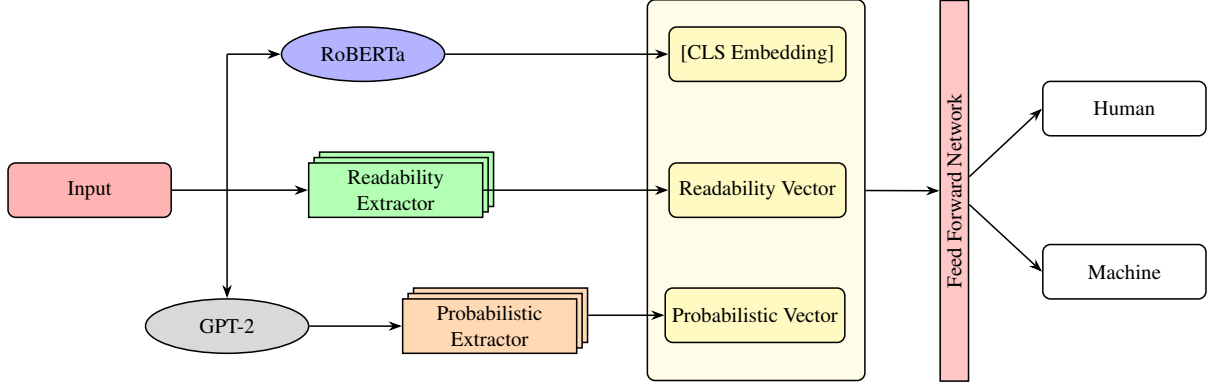


Figure 1: Overview of the proposed Hybrid Model Architecture combining three different feature vectors.

race", where newer LLMs are already far ahead of their predecessors. Although, zero-shot detectors are considered less accurate, innovations like DetectGPT, which only uses log probabilities computed by the model of interest, show that they can accelerate the development of more robust and generalizable methods of machine-generated text detection (Mitchell et al., 2023).

In the shared task, however, most prominently used by the participating teams were supervised training methods. Among those, there are three main groups of methods. The first one is a fine-tuned detection, which uses a pre-trained language model (PLM) and further trains it on the training dataset. Team USTC-BUPT is among the best performing teams at the monolingual track of Subtask A only utilizing PLMs. By adding a gradient reversal layer and a domain classifier that additionally outputs domain labels they achieved an accuracy of 96.10% (Guo et al., 2024).

A second major line focuses on statistical and probabilistic methods. These methods build on the principle that LLMs choose tokens they assign a higher probability to, while humans tend to write in a more "surprising" way. The first ranking Team Genaios employ their model LLMixtic, which mixes token-level probabilistic features from four different Llama-2 models forming a feature vector that is fed into a Transformer Encoder. Their method proved highly effective, achieving an accuracy of 96.88% on the monolingual and 75.67% on the multilingual track (Sarvazyan et al., 2024). Finally, there is a stylistic detection that analyzes linguistic features to distinguish between machine-generated and human written text. By testing a variety of different features as lexical diversity, text statistics and readability features and combining them with CLS embeddings from a transformer en-

coder model, Team PetKaz achieved an accuracy of 90.51% on the monolingual track (Petukhova et al., 2024).

3 Methodology

Figure 1 depicts the architecture of our best performing model. It combines three distinct types of features extracted from an input text. The semantic embeddings of a fine-tuned auto-encoder model, different readability metrics and token-level probabilistic features. This feature vector is passed through a feed-forward neural network for classification.

3.1 Semantic Feature Extraction

We employ RoBERTa-base, fine-tuned on the Machine-generated text classification task. The CLS Token represents the entire input sequence and is placed at the beginning of the input, capturing important semantic and contextual information. For each input we extract the CLS Token embedding, which for RoBERTa-base is 768-dimensional, and use it as the first part of our feature vector.

3.2 Readability Feature Extraction

While today's LLMs generate grammatically correct and coherent text, their output subtly differs from human written text, which we tried to make use of by devoting another part of our feature vector to a detailed linguistic analysis of the input sequence. Team Petkaz at the shared task, whose architecture strongly resembles ours, suggests a range of linguistic features such as text statistics like the number of difficult words, stylistic features, lexical diversity and readability features. Based on our data analysis, we decided to include the latter into our model. Previous works suggest that machine-generated text tends to be more

difficult to read, i.e more complex than human written equivalents. (Georgiou, 2024) for example, reports higher Flesch Reading Ease scores for human texts, which suggest that they are easier to read. Guided by this thesis, we calculate a variety of readability metrics including the following.

Flesch Reading Ease and Flesch-Kincaid Grade Level. Focus on sentence length and syllable count per word.

$$FK = 0.39 \times \frac{\text{total words}}{\text{total sentences}} + 11.8 \times \frac{\text{total syllables}}{\text{total words}} - 15.59$$

The FK Grade-Level was first used by the US army to assess the difficulty of technical manuals. With its weighing factors it outputs a U.S. grade level determining the texts difficulty.

Gunning Fog Index. Assesses sentence complexity by analyzing polysyllabic words.

$$GFI = 0.4 \times \left(\frac{\text{total words}}{\text{total sentences}} + 100 \times \frac{\text{complex words}}{\text{total words}} \right)$$

Words are considered as complex if they consist of three or more syllables. Common suffixes (such as -es, -ed or -ing) are not included. The output is the estimated years of education it requires to read a text of this complexity.

Automated Readability Index. Works like the GFI but instead of counting syllables it counts the number of characters per word and number of words per sentence. and outputs a score that maps to a Grade Level from Kindergarten to College Student.

$$ARI = 4.71 \times \frac{\text{characters}}{\text{words}} + 0.5 \times \frac{\text{total words}}{\text{total sentences}} - 21.43$$

3.3 Token-Probabilistic Feature Extraction

This part of the feature extraction focuses on statistical detection of machine-generated text and follows the core hypotheses of machine-generated text being more predictable than human-written text. While humans tend to write in a more surprising way, LLMs are optimized to choose the next most likely token (Holtzman et al., 2019). To capture these metrics we extract a comprehensive set of features from the token level probability distribution generated by GPT-2.

For a given input text with the tokens $x_1, x_2, \dots, x_n$, we process the sequence with GPT-2. At the i -th position we can extract the probability distribution $P(x_{i+1}|x_1, \dots, x_i)$ over the entire GPT-2 Vocabulary. From this distribution we can derive different features. Here, we used

features inspired by the work of Genaios, the first ranking team in the shared task and got a feature vector including the following metrics, averaged across the entire sequence.

For each position we calculate (i) the log probability of the observed token, measuring how likely the token is, given the tokens already seen. For the feature vector we then use the average observed token probability which essentially describes the perplexity of the input sequence. Perplexity is defined as the exponentiated average negative log-likelihood of a sequence X with t tokens:

$$ppl(X) = \exp \left(-\frac{1}{t} \sum_i \log p_\theta(x_i|x_{<i}) \right)$$

Intuitively, it can be thought of as an evaluation of the model’s uncertainty when predicting the next token. A lower perplexity, and thus a higher average observed log-probability, is a strong indicator of machine-generated text. We further calculate (ii) the probability of the most likely token, which describes the confidence of the model at each step. Finally, we take (iii) the entropy, which measures the uncertainty of the distribution.

$$\beta_i = - \sum_{y \in \mathcal{V}} p_\theta(y|x_{<i}) \log p_\theta(y|x_{<i})$$

A low entropy would indicate that the model is highly confident in a few tokens.

3.4 Feed-forward neural network

The concatenated feature vector is then passed into a feed-forward neural network, where the output layer classifies the text as either human or machine.

3.5 Multilingual

Transferring our previously described system architecture to a multilingual context presented significant challenges. Considering that GPT-2 is a transformer model pre-trained on a very large corpus of English data only, the feature representation from the token-probabilistic feature extractor was of no use for non-english texts. Moreover, readability features are quite language-specific. Applying the same feature for various languages yields very inaccurate to irrelevant results. To address these challenges, our multilingual approach shifts to a more basic but very effective system relying on a fine-tuned XLM-RoBERTa (XLM-R) model. XLM-R is a model trained on 100 different languages, which makes it suitable for cross-lingual

tasks. Unlike in the monolingual system, we do not apply a self-designed feed-forward neural network, but utilize the transformer library that automatically attaches a classification head on top of the model that produces the final classification logits.

4 Experiments

4.1 Dataset and Evaluation

We used the SemEval-2024 Task 8 dataset, which is an extension of the M4 dataset (Wang et al., 2023). The training set includes five domains: Wikipedia, WikiHow, Reddit, arXiv and PeerRead. Machine-generated texts were produced by various LLMs, including davinci-003, ChatGPT, Cohere and Dolly-v2. For the test set Outfox was used as a surprising domain and GPT-4 and BLOOMz are introduced as surprising generators. The labels are roughly balanced with 56,400 machine-generated and 63,351 human-written texts. The multilingual training set additionally contains texts in Chinese, Urdu, Bulgarian and Indonesian, and for the multilingual test set, Arabic, German and Italian are introduced as unseen languages. Table 1 depicts the dataset statistics. As in the official shared task setting, we adhered to the rules of not using additional training data, and by treating the test set labels as unavailable during the development of our model in order to assure a fair comparison to other teams results. Following the official task description, we employed accuracy as the evaluation metric to determine the quality of our model.

Track	#Train	#Dev	#Test
Monolingual	119,757	5000	34,272
Multilingual	172,417	4000	42,378

Table 1: Dataset statistics for Subtask A

4.2 Experimental Setup

We use RoBERTa-base as the source of the CLS embeddings. For the fine-tuning process we used the baseline framework provided in the task’s description, and slightly modified to support parameter-efficient fine-tuning (PEFT). With limited computing resources at hand, we were searching for ways, to maximize performance, while minimizing computational costs. Low-Rank Adaptation (LoRA) is reported to achieve comparably good or better results than full fine-tuning, despite having fewer trainable parameters. By freezing pre-trained model weights

and adding small, trainable matrices to the model’s layers, LoRA drastically reduces the number of trainable parameters (Hu et al., 2022). We trained RoBERTa-base for 3 epochs with a learning rate of $2e-4$ and a batch size of 16. XLM-R was trained for 4 epochs with a learning rate of $2e-5$. For the LoRa configurations, we set the rank of the low-rank matrices to 64, lora_alpha (scaling factor of the learning rate) at 16 and a dropout of 0.01 to avoid overfitting. The feed-forward neural network consists of three fully connected hidden layers, followed by a batch normalization, a ReLU activation function and a dropout layer with a rate of 0.3 to reduce overfitting and improve the generalization of the model. The weights were initialized using Xavier/Glorot initialization to avoid vanishing or exploding gradients.

All our models were trained using the NVIDIA A100 GPU. We used the Hugging Face Transformer library to fine-tune RoBERTa and XLM-RoBERTa, and developed our final model using PyTorch.

4.3 Experiments on the development set

Table 2 shows the evaluation results on the development set with different combinations of our hybrid model. As a baseline we used the fine-tuned LoRA RoBERTa.

Configuration	Accuracy	F1-Score
emb	0.744	0.727
emb + lex	0.70	0.675
emb + read	0.773	0.76
emb + prob	0.727	0.72
emb + read + prob	0.757	0.75

Table 2: Accuracy of different configurations on the development set of the monolingual track. Configurations include embeddings (emb), lexical diversity (lex), readability (read), token-probabilistic (prob)

First of all, we see that the best accuracy of 77.3% is achieved by only adding readability features to the RoBERTa baseline. With a slight decrease of 1.6% in accuracy, the combination of readability and probabilistic features is the second best performing configuration. We note, however, that both configurations increase the accuracy compared to a transformer encoder only approach. Adding only probabilistic features, however, causes a slight decrease in performance with an accuracy of only 71.02%.

Apart from readability, we first also experimented with other linguistic features. (Petukhova et al., 2024) reports that their best accuracy is achieved by combining RoBERTa embeddings with lexical diversity features including type-token ratio and hypergeometric distribution d (HDD). When employing this feature combination, however, we noted a decrease in accuracy compared to the RoBERTa only configuration. Upon further investigation, we noticed that machine-generated texts in the training data are associated with a higher TTR than human texts. In the development set, however, it is the other way around. This might be because of the surprising generator BLOOMz that was used for the dev set. For Team PetKaz, it might work because they use a reduced training set, which improves the accuracy of this specific configuration from 0.73% to 0.95%. We, however, decided that lexical diversity might not be robust enough across different generators to serve as a reliable feature for the classification task. While our model variant that only uses readability features with embeddings performed best on the development set, we ultimately decided to choose the second best variant that also includes probabilistic features as our final model. This decision is based on the assumption that generally probabilistic features provide valuable information as seen in the work from Team Genaios that can make our model more robust and achieve better results in the test set where a wider variety of LLMs are used for the generation of AI texts.

Based on our results with the monolingual LoRA-RoBERTa, we hypothesized that following the same approach in Parameter-Efficient Fine-Tuning would achieve good results on the multilingual track by fine-tuning XLM-R with the same LoRA parameters. However, initial results obtained on the development set were not satisfactory, which is why we decided to do a full fine-tuning of XLM-R. This ultimately led to a performance increase of approximately 8%.

Model	Accuracy	F1-Score
LoRA XLM-R	0.5493	0.482
XLM-R	0.6295	0.603

Table 3: Accuracy of LoRA XLM-R vs. fully fine-tuned XLM-R on multilingual development set

4.4 Data Augmentation

During model training, we aim to improve the accuracy of machine-generated text detection through data augmentation. Overfitting is a common phenomenon, where the model achieves high accuracy on the training data but performs poorly on unseen test data. A simple yet effective approach to mitigate this issue is to enlarge the training dataset, thereby optimizing the final model by replacing or extending the original training samples.

To enhance the diversity of the training data, we applied data augmentation exclusively to human-written samples (label = 0) in the training set. The objective was to introduce linguistic variation into the human class, thereby reducing overfitting and improving the model’s ability to generalize in distinguishing human-written from machine-generated text.

For this purpose, we employed the `ibm-research/qcpg-sentences` paraphrasing model (Bandel et al., 2022), a pre-trained sequence-to-sequence model designed for sentence-level rewriting. All human-written texts were first segmented into sentences using punctuation and line breaks as delimiters. We then compiled a set of unique sentences and paraphrased them in batches of 256 using the model on GPU. In total, the augmentation process produced a large collection of paraphrased human-written samples. This procedure effectively expanded the training set and increased the variability of human text without altering the machine-generated samples, thereby balancing the data in a way that favors improved generalization of the detection model.

In the preliminary evaluation using only the baseline RoBERTa model, the accuracy improved from 65.14% with the original training data to 68.10% after applying data augmentation. However, when the paraphrased sentences were incorporated into the dataset by replacing the original training samples and applied to the hybrid model, no improvement in accuracy was observed; the performance remained at only 75%. This indicates that simply enlarging the dataset does not necessarily lead to better model performance. A possible explanation is that the paraphrased sentences did not reinforce or enrich the discriminative features available to the model and may even have blurred the boundary between human-written and machine-generated text, since paraphrasing itself can be considered a form of assisted generation.

Therefore, we did not retain data augmentation in the final implementation. Nevertheless, this finding suggests that in future stages, data augmentation may be more effective if it relies on more “manually collected” data resources, ensuring that the distinctive features of human-written text are preserved, and by selecting augmentation strategies tailored to different model architectures.

References

- Elron Bandel, Ranit Aharonov, Michal Shmueli-Scheuer, Ilya Shnayderman, Noam Slonim, and Liat Ein-Dor. 2022. Quality controlled paraphrase generation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 596–609, Dublin, Ireland. Association for Computational Linguistics.
- Georgios P Georgiou. 2024. Differentiating between human-written and ai-generated texts using linguistic features automatically extracted from an online computational tool. *arXiv preprint arXiv:2407.03646*.
- Zikang Guo, Kaijie Jiao, Xingyu Yao, Yuning Wan, Haoran Li, Benfeng Xu, Licheng Zhang, Quan Wang, Yongdong Zhang, and Zhendong Mao. 2024. Ustc-bupt at semeval-2024 task 8: Enhancing machine-generated text detection via domain adversarial neural networks and llm embeddings. In *Proceedings of the 18th International Workshop on Semantic Evaluation (SemEval-2024)*, pages 1511–1522.
- Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. 2019. The curious case of neural text degeneration. *arXiv preprint arXiv:1904.09751*.
- Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, and 1 others. 2022. Lora: Low-rank adaptation of large language models. *ICLR*, 1(2):3.
- Lama Ahmad Jan Hendrik Kirchner. 2023. New ai classifier for indicating ai-written text. <https://openai.com/index/new-ai-classifier-for-indicating-ai-written-text/>. Accessed on 19.08.2025.
- Sarah Kreps, R Miles McCain, and Miles Brundage. 2022. All the news that’s fit to fabricate: Ai-generated text as a tool of media misinformation. *Journal of experimental political science*, 9(1):104–117.
- Eric Mitchell, Yoonho Lee, Alexander Khazatsky, Christopher D Manning, and Chelsea Finn. 2023. Detectgpt: Zero-shot machine-generated text detection using probability curvature. In *International conference on machine learning*, pages 24950–24962. PMLR.
- Kseniia Petukhova, Roman Kazakov, and Ekaterina Kochmar. 2024. Petkaz at semeval-2024 task 8: Can linguistics capture the specifics of llm-generated text? *arXiv preprint arXiv:2404.05483*.
- Areg Mikael Sarvazyan, José-Ángel González, and Marc Franco-Salvador. 2024. Genaios at semeval-2024 task 8: Detecting machine-generated text by mixing language model probabilistic features. In *Proceedings of the 18th International Workshop on Semantic Evaluation (SemEval-2024)*, pages 101–107.
- Jan Philip Wahle, Terry Ruas, Frederic Kirstein, and Bela Gipp. 2022. How large language models are transforming machine-paraphrased plagiarism. *arXiv preprint arXiv:2210.03568*.
- Yuxia Wang, Jonibek Mansurov, Petar Ivanov, Jinyan Su, Artem Shelmanov, Akim Tsvigun, Osama Mohammed Afzal, Tarek Mahmoud, Giovanni Puccetti, Thomas Arnold, and 1 others. 2024. Semeval-2024 task 8: Multidomain, multimodel and multilingual machine-generated text detection. *arXiv preprint arXiv:2404.14183*.
- Yuxia Wang, Jonibek Mansurov, Petar Ivanov, Jinyan Su, Artem Shelmanov, Akim Tsvigun, Chenxi Whitehouse, Osama Mohammed Afzal, Tarek Mahmoud, Toru Sasaki, and 1 others. 2023. M4: Multi-generator, multi-domain, and multi-lingual black-box machine-generated text detection. *arXiv preprint arXiv:2305.14902*.